INFORME TÉCNICO · ANDROID

PROYECTO ACADÉMICO

Explorador Pokédex

Estructura del proyecto, arquitectura y funcionamiento

Aplicación móvil para consultar y explorar Pokémon usando PokéAPI. Desarrollada en Kotlin con Jetpack Compose, Retrofit y una arquitectura separada en presentación, dominio y datos.

Fuente de datos
API REST pública PokéAPI v2

Interfaz
Jetpack Compose y Material 3

Patrón
MVVM con repositorio por capas

Plataforma
Android · Kotlin · API mínima 24

Documento preparado a partir del código y la configuración actuales del repositorio.

01 · VISIÓN GENERAL

¿Qué hace la aplicación?

Explorador Pokédex consulta el catálogo de PokéAPI y permite buscar Pokémon por nombre o número, filtrar por tipo y abrir una pantalla independiente con información ampliada. Los resultados se cargan por páginas al desplazarse. La interfaz informa el estado de conectividad y presenta estados de carga y mensajes de error con opción de reintento.

Estructura principal

```
PokedexExplorerApp/
├── app/
│   ├── build.gradle.kts                # Android, Compose y dependencias
│   └── src/main/
│       ├── AndroidManifest.xml         # Actividad, nombre, icono y permisos
│       ├── res/                         # Iconos y recursos visuales
│       └── java/com/luistorres/pokedexexplorer/
│           ├── MainActivity.kt
│           ├── data/
│           │   ├── network/ConnectivityObserver.kt
│           │   ├── remote/              # Retrofit, servicio y DTO JSON
│           │   └── repository/           # Consultas y mapeo de respuestas
│           ├── domain/
│           │   ├── model/Pokemon.kt
│           │   └── repository/PokemonRepository.kt
│           └── presentation/
│               ├── PokedexApp.kt        # Catálogo y pantalla de detalle
│               ├── PokemonViewModel.kt  # Estado y operaciones de UI
│               └── theme/Theme.kt
├── docs/ENDPOINTS_Y_ARQUITECTURA.md
├── INFORME_PROYECTO.md
├── README.md
├── build.gradle.kts
└── settings.gradle.kts
```

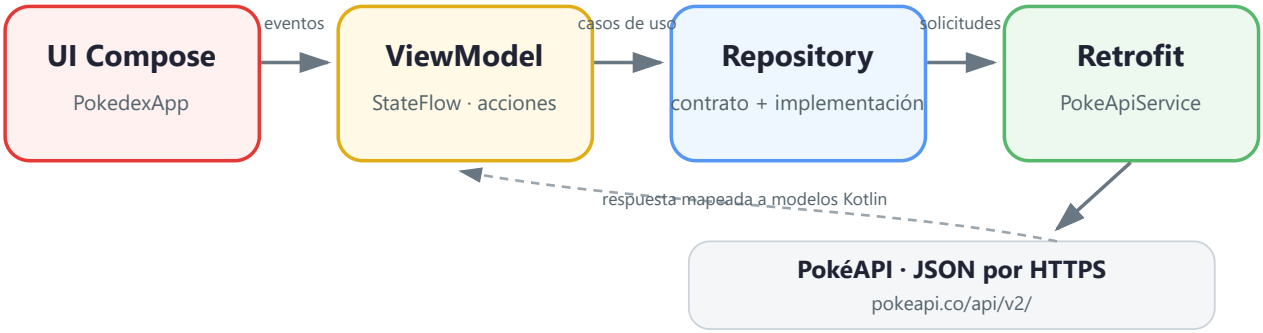
Responsabilidad de cada carpeta

Parte	Responsabilidad	Archivos representativos
presentation	Pantallas Compose, navegación y estado observable.	`PokedexApp.kt`, `PokemonViewModel.kt`
domain	Modelos usados por la aplicación y contrato del repositorio.	`Pokemon.kt`, `PokemonRepository.kt`
data	Acceso remoto, observación de red, DTO y conversión al dominio.	`PokeApiService.kt`, `PokemonRepositoryImpl.kt`

> **Separación:** la UI no interpreta respuestas JSON ni conoce detalles de Retrofit. El repositorio se encarga de consultar la API y transformar los datos.

02 · ARQUITECTURA

Capas y flujo de datos



Recorrido de una consulta

1. El usuario toca un filtro, escribe una búsqueda, solicita otra página o selecciona una tarjeta.
2. La pantalla comunica la acción al `PokemonViewModel`.
3. El ViewModel ejecuta la operación suspendida del contrato `PokemonRepository` y actualiza un `StateFlow<PokemonUiState>`.
4. `PokemonRepositoryImpl` pide recursos a `PokeApiService` y mapea los DTO a modelos de dominio.
5. La UI observa el nuevo estado y Compose vuelve a dibujar los componentes afectados.

Modelo MVVM

Model: modelos de dominio Pokémon, estadísticas y detalles de especie. **View:** componentes y pantallas Compose. **ViewModel:** coordina la UI, la red y estados como carga, error, búsqueda y conexión.

> **Inyección:** el repositorio se construye actualmente desde el ViewModel con `RetrofitProvider`. Hilt figura como recomendación en la actividad, pero no está configurado en el proyecto.

03 · FUENTE REMOTA

PokéAPI y conversión de datos

La URL base se configura en `RetrofitProvider.kt`. Todas las solicitudes actuales usan el método GET y reciben JSON. Retrofit define la interfaz del servicio; Gson convierte las respuestas a DTO declarados en `ApiModels.kt`.

Endpoint	Parámetros	Respuesta usada	Función en la app
`GET /pokemon`	Query: `limit=24`, `offset=0...`	`count`, `next`, `previous`, `results[]`	Catálogo y paginación.
`GET /pokemon/{nameOrId}`	Ruta: nombre o número.	Datos del Pokémon: sprites, tipos, habilidades y estadísticas.	Buscar por nombre o número y recuperar el recurso completo asociado a cada resultado.
`GET /type/{type}`	Ruta: identificador del tipo, por ejemplo `fire`.	Lista `pokemon[]` de ese tipo.	Filtrar por tipo; luego se consultan sus recursos Pokémon.
`GET /pokemon-species/{id}`	Ruta: número de Pokédex.	Indicadores legendario/mítico y `flavor_text_entries[]`.	Descripción en español y clasificación de especie.
`GET /ability/{name}`	Ruta: identificador de la habilidad.	`names[]` con nombre e idioma.	Nombre localizado de la habilidad, si hay traducción al español.

Mapeo entre respuesta y dominio

El endpoint de listado solo proporciona nombres y URL de referencia. Por cada nombre, el repositorio consulta `/pokemon/{name}`, y construye un `Pokemon` con id, nombre, ilustración, altura, peso, tipos, habilidades y estadísticas. En la ficha, el endpoint de especie añade descripción y marcas de legendario/mítico; los endpoints de habilidad aportan nombres localizados. Si una habilidad no tiene traducción disponible, se muestra el aviso correspondiente.

La descripción selecciona la entrada cuyo idioma tiene el código `es` y normaliza los espacios. Los sprites usan primero la ilustración oficial y, si falta, el sprite frontal estándar.

> **Concurrencia:** el repositorio utiliza coroutines y `async/awaitAll` para solicitar los detalles de varios Pokémon en paralelo dentro de una página o filtro.

04 · EXPERIENCIA DE USO

Funciones principales

Catálogo y scroll infinito

La app solicita 24 resultados desde `/pokemon`, junto con los detalles individuales necesarios para las tarjetas. La interfaz observa qué elemento de la cuadrícula está visible. Al aproximarse al final, el ViewModel pide la página siguiente con un `offset` nuevo. El campo `next` de la respuesta determina si quedan páginas.

Búsqueda y filtros

- La búsqueda normaliza el texto y consulta `/pokemon/{nameOrId}`; acepta nombre o número de Pokédex.
- El filtro por tipo consulta `/type/{type}` y carga los recursos asociados. La pantalla ofrece tipos como planta, fuego, agua, eléctrico, bicho, normal, veneno, hada, dragón, lucha, psíquico, roca, fantasma, hielo, siniestro, acero, volador y tierra.
- La opción “Todos” vuelve a solicitar el catálogo paginado.

Navegación y pantalla de detalle

Navigation Compose mantiene una ruta para el catálogo y una ruta parametrizada `pokemon/{pokemonId}`. Al tocar una tarjeta, la aplicación abre la pantalla del Pokémon elegido. La pantalla consulta los datos por identificador y los complementa con especie y habilidades.

La ficha presenta ilustración, número, nombre, tipos, descripción, medidas, habilidades, estadísticas base y una marca para Pokémon legendarios o míticos. Incluye navegación de regreso, estado de carga y reintento si falla la consulta.

Idioma y presentación

Los textos de interfaz, mensajes y etiquetas de tipo/estadística están en español. Los nombres propios de Pokémon se muestran según el identificador canónico de la API; la disponibilidad de descripciones y nombres de habilidades traducidos depende de PokéAPI.

05 · ESTADO Y CONFIABILIDAD

Conectividad, errores y estado

Estado de conexión

`ConnectivityObserver` utiliza Android `ConnectivityManager` y `NetworkCapabilities`. Publica cambios de conectividad como un Flow. El ViewModel recoge ese flujo y la UI muestra “En línea” o “Sin conexión”. La aplicación también necesita conectividad para consultar PokéAPI y descargar ilustraciones.

Estados de pantalla

Estado	Comportamiento
Carga inicial / búsqueda / filtro	Indicador de progreso mientras se consultan datos.
Carga de página adicional	Indicador al final del catálogo.
Error sin resultados	Mensaje en español y acción “Intentar de nuevo”.
Error al cargar más	Mensaje junto al catálogo con opción de reintentar.
Error de detalle	La ficha conserva los datos básicos disponibles y permite repetir la solicitud.

Modelos principales

- `Pokemon`: id, nombre, URL de imagen, altura, peso, tipos, habilidades, estadísticas y datos complementarios.
- `PokemonStat`: nombre y valor de una estadística base.
- `PokemonSpeciesDetails`: descripción y marcas de Pokémon legendario/mítico.
- `PokemonUiState`: resultados, carga, error, filtro actual, búsqueda, conectividad, paginación y detalle seleccionado.

Límites conocidos

- No hay persistencia local; el catálogo se consulta en línea.
- El filtro por tipo muestra hasta 40 resultados por solicitud.
- Los nombres de especies y descripciones traducidos dependen de las entradas que proporcione PokéAPI.
- La conexión se observa por capacidad de Internet del sistema; esto no garantiza que el servidor remoto esté disponible.

06 · HERRAMIENTAS Y EJECUCIÓN

Stack tecnológico

Tecnología	Uso
Kotlin	Lenguaje de implementación.
Jetpack Compose + Material 3	Interfaz declarativa, componentes y diseño adaptable.
Navigation Compose	Navegación entre catálogo y detalle.
ViewModel + StateFlow	Estado de pantalla y supervivencia ante recomposiciones.
Retrofit + Gson	Solicitudes HTTP y mapeo JSON a DTO.
Kotlin Coroutines	Operaciones asíncronas y llamadas concurrentes.
Coil	Carga de ilustraciones remotas en Compose.

Configuración Android

El módulo de app declara `compileSdk 35`, `minSdk 24`, `applicationId com.luistorres.pokedexexplorer` y objetivo Java 17. El manifiesto declara permiso de Internet y el nombre de la app. Los iconos están en `app/src/main/res`.

Compilación y comprobación

El APK de depuración se genera ejecutando `:app:assembleDebug` con Gradle desde la raíz del proyecto. El archivo de salida es `app/build/outputs/apk/debug/app-debug.apk`. ADB permite instalarlo en el emulador y abrir el paquete `com.luistorres.pokedexexplorer`.

En el entorno de desarrollo se compiló el APK correctamente, se instaló en el emulador y se comprobó visualmente la carga del catálogo y la navegación a la ficha de Bulbasaur con datos recibidos de PokéAPI.

Uso de inteligencia artificial

Se utilizó ChatGPT con el agente de programación Codex como apoyo para analizar los requisitos, revisar la arquitectura existente, implementar y pulir la pantalla navegable de detalle, manejar estados de error y preparar documentación. El flujo consistió en describir la tarea, revisar los cambios en el código, compilar con Gradle, usar los mensajes del compilador para corregir y comprobar el resultado en el emulador mediante ADB.

No se utilizaron otros agentes autónomos para este trabajo ni IA para generar los datos de Pokémon. La fuente de esos datos es PokéAPI. Las respuestas de IA se trataron como asistencia y se contrastaron con el código, la compilación y la ejecución.

Referencias: Documentación PokéAPI v2 · Jetpack Compose · Arquitectura Android.